

Computer Science (Degree)

CTSE 2025 - Descriptive Paper II - Questions with Answers

- Maximum marks: 100 Duration: 180 minutes
- Booklet 2537_25 (II), Series A
- Section A: answer any 8 of Q1-Q10 (8 x 5 = 40 marks)
- Section B: answer any 3 of Q11-Q15 (3 x 10 = 30 marks)
- Section C: answer any 2 of Q16-Q19 (2 x 15 = 30 marks)
- Subject areas: operating systems, software engineering and project management, databases, computer networks, web technology, artificial intelligence

How to use this document

- Every question is followed by a red spoiler bar and a page break. The answer starts on the next page, so you cannot see it while reading the question.
- Answer length is scaled to the marks: roughly 5 marks = 5-6 points, 10 marks = a worked explanation, 15 marks = full treatment.
- Where a diagram or a table is expected, the document lays out exactly what to draw.
- For underlying theory that is not specific to one question, see the companion document: CTSE 2025 - Concepts and Revision Notes.docx

Section A - Q1 to Q10 - 5 marks each (answer any 8)

Q1 (5 marks)

What are the key components of an operating system? Briefly explain the function of each.

▼ ANSWER BELOW ▼

Stop here if you are still attempting this question.

Answer to Q1

An operating system is the software layer between the hardware and the user's programs. It manages resources and provides an abstract, convenient interface to them. Its major components are the following.

- Kernel - the core that always resides in memory. It has the highest privilege level and mediates all access to hardware. Everything below is either part of the kernel or built on it.
- Process management - creates, schedules, suspends, resumes and terminates processes and threads. It maintains a Process Control Block for each process, performs context switching, and provides inter-process communication and synchronisation primitives such as semaphores and mutexes. It also detects or prevents deadlock.
- Memory management - tracks which parts of memory are in use and by whom, allocates and deallocates space, translates logical addresses to physical addresses through the MMU, and implements virtual memory using paging or segmentation with swapping and page replacement.
- File system management - organises data into files and directories, controls creation, reading, writing and deletion, maps files onto disk blocks, manages free space, and enforces access permissions.
- Device management, or the I/O subsystem - controls peripherals through device drivers, provides a uniform interface to dissimilar hardware, and improves throughput with buffering, caching and spooling.
- Secondary storage management - allocates disk space, schedules disk requests with algorithms such as SCAN and C-SCAN, and handles partitioning and formatting.
- Protection and security - authenticates users, enforces access rights using access control lists or capabilities, isolates one process's address space from another's, and maintains audit logs.
- Command interpreter, or shell - accepts user commands, whether through a command line or a graphical shell, and starts the corresponding programs.

One extra point for full marks

The system-call interface is what binds these components to user programs: a program requests a service such as open, read, fork or exec, the CPU traps into kernel mode, the kernel performs the operation on the program's behalf and returns. The distinction between user mode and kernel mode is the hardware feature that makes protection possible.

Q2 (5 marks)

Differentiate between logical and physical addresses in memory management.

▼ **ANSWER BELOW** ▼

Stop here if you are still attempting this question.

Answer to Q2

Logical address, also called virtual address

- The address generated by the CPU while a program executes. It is what the program itself sees and what appears in the compiled instructions.
- It refers to a location in the process's own logical address space, which always begins at 0 regardless of where the process actually sits in memory.
- The set of all logical addresses a process can generate is its logical address space.
- A user program deals only with logical addresses; it never sees a physical one.

Physical address

- The actual address presented on the memory bus to select a location in main memory.
- It refers to a real location in RAM; the set of all such addresses is the physical address space.
- It is loaded into the memory address register of the memory unit.

How the translation happens

- The Memory Management Unit, a hardware component, performs the mapping at run time - it must be hardware, because it happens on every single memory reference.
- With simple relocation: $\text{physical address} = \text{logical address} + \text{contents of the relocation, or base, register}$. A limit register is checked at the same time so that an address beyond the process's allocation raises a trap, which is how one process is prevented from touching another's memory.
- With paging: the logical address splits into a page number and an offset. The page number indexes the page table to obtain a frame number, and $\text{physical address} = \text{frame number} \times \text{page size} + \text{offset}$. The TLB caches recent translations to avoid a second memory access per reference.

Why the distinction matters

- Relocation - a program can be loaded anywhere, or moved, without recompiling or relinking, because only the mapping changes.
- Protection - a process cannot express an address outside its own space, so isolation is enforced by the hardware.
- Virtual memory - the logical space may be larger than the physical memory available, with the remainder held on disk and brought in on demand.
- Sharing - two processes can map the same physical frame at different logical addresses, which is how shared libraries and shared memory work.

In compile-time and load-time binding the logical and physical addresses are identical. They differ only with execution-time binding, which is what every modern general-purpose operating system uses.

Q3 (5 marks)

What are the different phases in a typical Software Development Life Cycle (SDLC)? Write a brief note on each of them.

▼ **ANSWER BELOW** ▼

Stop here if you are still attempting this question.

Answer to Q3

The SDLC is the structured sequence of activities through which software is conceived, built, delivered and maintained. The classical phases are the following.

- Requirement gathering and analysis - interact with stakeholders through interviews, questionnaires and observation to establish what the system must do. Separate functional requirements, what the system does, from non-functional requirements, how well it does it. Study feasibility - technical, economic, legal and operational. The output is the Software Requirements Specification, the SRS, which becomes the contract between customer and developer.
- System design - decide how the requirements will be met. High-level design fixes the architecture, the modules and their interfaces, the database schema and the technology stack; low-level design specifies the internal logic of each module, its algorithms and its data structures. The output is the design document, often expressed with UML diagrams.
- Implementation, or coding - developers write the source code module by module according to the design, following coding standards, and perform unit testing as they go. The output is the working source code and the built executable.
- Testing - verify that the software matches the SRS and find defects. Unit testing checks individual components, integration testing checks the interfaces between them, system testing checks the whole product against the requirements, and acceptance testing is performed by the customer. The output is the test report and defect log.
- Deployment - install the software in the live environment, migrate data, configure the servers, train the users, and cut over from any earlier system, whether directly, in parallel, or in phases.
- Maintenance - the longest and most expensive phase, typically the majority of total lifetime cost. It covers corrective maintenance to fix defects found in use, adaptive maintenance to keep pace with changes in the environment, perfective maintenance to add enhancements, and preventive maintenance to reduce future decay.

Points worth adding

- The phases are the same across process models; what differs is how they are sequenced. The waterfall model runs them once in strict order; iterative and incremental models repeat them over successive releases; the spiral model repeats them with risk analysis at the start of each cycle; Agile compresses them into short sprints of one to four weeks.
- The cost of fixing a defect grows sharply the later it is found, which is the economic argument for investing effort in requirements and design.

Q4 (5 marks)

Explain the purpose of a Work Breakdown Structure (WBS) in software project management.

▼ **ANSWER BELOW** ▼

Stop here if you are still attempting this question.

Answer to Q4

A Work Breakdown Structure is a hierarchical, deliverable-oriented decomposition of the total scope of a project into progressively smaller components, continuing until each component is a work package small enough to be estimated, assigned and tracked.

Purposes it serves

- Scope definition and control - the WBS defines the whole of the work and, by implication, everything outside it. Anything not in the WBS is not in the project, which is the primary defence against scope creep.
- Makes estimation possible - estimating a whole project in one step is unreliable; estimating many small work packages and rolling the figures up is far more accurate. This is bottom-up estimation.
- Assignment of responsibility - each work package can be given to one person or team, which removes ambiguity about ownership. Combined with a responsibility assignment matrix, it maps work to people.
- Basis for scheduling - the work packages become the activities of the network diagram, from which dependencies, the critical path and the Gantt chart are derived.
- Basis for cost budgeting and control - cost is assigned per work package to form the cost baseline, and earned value analysis then measures progress against it.
- Progress measurement - completion of a small work package is objective, either done or not done, which avoids the familiar problem of a task reported as ninety per cent complete for weeks.
- Risk identification - decomposing the work exposes the packages that are technically uncertain, so risk can be attached where it belongs.
- Communication - it gives the team, the customer and management a single shared picture of the work.

How it is constructed

- Decomposition may be by deliverable, by phase, or by subsystem. Deliverable-oriented decomposition is generally preferred.
- The hundred per cent rule: the children of any node must account for exactly all of the work of that node, no more and no less.
- Work packages are conventionally sized so that they take between eight and eighty hours, and no longer than one reporting period.
- Every element gets a unique code, for example 1.2.3, used consistently in the schedule, the budget and the reports.
- A WBS dictionary accompanies it, describing each work package, its acceptance criteria and its assumptions.

A WBS shows what must be produced, not when or by whom or in what order. Sequence belongs in the schedule, and the WBS should list deliverables as nouns rather than activities as verbs.

Q5 (5 marks)

What are the key characteristics of a Relational Database Management System (RDBMS)?

▼ **ANSWER BELOW** ▼

Stop here if you are still attempting this question.

Answer to Q5

An RDBMS stores data according to the relational model proposed by E. F. Codd in 1970, in which all data is represented as relations, that is as tables of rows and columns.

Key characteristics

- Data is held in tables. A relation is a table; a row is a tuple or record; a column is an attribute or field. The number of columns is the degree and the number of rows is the cardinality.
- Every value is atomic. A cell holds a single indivisible value, not a list or a repeating group. This is first normal form.
- Rows are unordered and uniquely identifiable. There is no inherent row order, and every row is distinguished by its primary key rather than by position.
- Relationships are expressed by values, not by pointers. A foreign key in one table holds the primary-key value of a row in another, so the tables are joined logically at query time.
- Integrity constraints are enforced by the system. Domain constraints restrict a column to valid values and types; entity integrity requires a primary key that is unique and not null; referential integrity requires every foreign key to match an existing primary key or be null; and user-defined constraints capture business rules.
- Access is through a declarative language, SQL. The user states what is wanted, not how to obtain it, and the query optimiser chooses the execution strategy.
- Transactions are ACID. Atomicity means all or nothing; Consistency means the database moves from one valid state to another; Isolation means concurrent transactions do not interfere; Durability means a committed change survives a crash.
- Data independence. Logical data independence allows the conceptual schema to change without altering applications, and physical data independence allows the storage structures to change without altering the conceptual schema. This is the three-schema architecture.
- Normalisation is supported and encouraged, reducing redundancy and eliminating insertion, update and deletion anomalies.
- Security and multi-user control. Access is granted per user and per object with GRANT and REVOKE, views restrict visibility, and concurrency control with locking or multi-version techniques allows many users at once.

Examples

Oracle, MySQL, PostgreSQL, Microsoft SQL Server, IBM Db2 and SQLite.

Q6 (5 marks)

Explain the concept of a primary key and a foreign key in database design.

▼ **ANSWER BELOW** ▼

Stop here if you are still attempting this question.

Answer to Q6

Primary key

- An attribute, or a combination of attributes, that uniquely identifies each row of a table.
- It must be unique across all rows and must never be NULL, since a null value could not identify anything. Together these two rules are entity integrity.
- A table has exactly one primary key, though several candidate keys may exist; the one chosen is the primary key and the others become alternate keys.
- It should be stable, that is its value should not change, because other tables depend on it.
- A primary key spanning more than one column is a composite key. A key with no business meaning, added purely to identify rows, such as an auto-increment id, is a surrogate key.
- The RDBMS normally creates a unique index on it automatically, so lookups by primary key are fast.

Foreign key

- An attribute, or set of attributes, in one table whose values must match the primary-key values of another table, or be NULL. It is the mechanism that expresses a relationship between two tables.
- The table holding the foreign key is the child, or referencing, table; the table holding the matching primary key is the parent, or referenced, table.
- The rule that every non-null foreign-key value must correspond to an existing parent row is referential integrity. It prevents orphan rows, such as an order that refers to a customer who does not exist.
- A foreign key need not be unique - many rows may share one value, which is exactly how a one-to-many relationship is represented.
- A table may have several foreign keys, and a foreign key may reference its own table, which is how hierarchies such as employee-and-manager are stored.

Referential actions

When a referenced row is deleted or its key updated, the database applies the declared action: CASCADE propagates the change to the children, SET NULL clears the child's foreign key, SET DEFAULT substitutes a default value, RESTRICT or NO ACTION rejects the operation.

Example

```
CREATE TABLE Customer (  
    cust_id    INT PRIMARY KEY,  
    name       VARCHAR(60) NOT NULL  
);  
  
CREATE TABLE Orders (  
    order_id   INT PRIMARY KEY,  
    order_dt   DATE,  
    cust_id    INT,  
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id)  
        ON DELETE CASCADE  
);
```

Here cust_id identifies each customer uniquely, and Orders.cust_id records which customer placed each order. One customer may have many orders, so the relationship is one-to-many.

Q7 (5 marks)

What is the OSI model? Briefly list the seven layers.

▼ **ANSWER BELOW** ▼

Stop here if you are still attempting this question.

Answer to Q7

The Open Systems Interconnection model is a seven-layer reference architecture published by the International Organization for Standardization in 1984. It divides network communication into layers, each providing services to the layer above and using the services of the layer below. It is a conceptual reference, not an implementation - the Internet actually runs on the TCP/IP model - but it remains the standard vocabulary for describing networks and for isolating faults.

The seven layers, from the bottom up

- Layer 1, Physical - transmits raw bits over the medium. Concerned with voltage levels, pin layout, cable and connector specifications, data rate, and the physical topology. Data unit: bit. Devices: hub, repeater, cable, connector.
- Layer 2, Data Link - provides node-to-node delivery on the same network. Performs framing, physical addressing with MAC addresses, error detection with a CRC, flow control and media access control. It has two sublayers, LLC and MAC. Data unit: frame. Devices: switch, bridge, network interface card.
- Layer 3, Network - delivers packets from source host to destination host across multiple networks. Performs logical addressing with IP addresses, routing, path determination and fragmentation. Data unit: packet or datagram. Device: router. Protocols: IP, ICMP, ARP, OSPF.
- Layer 4, Transport - provides process-to-process delivery, identified by port numbers. Handles segmentation and reassembly, end-to-end error recovery and flow control, and offers either a connection-oriented reliable service or a connectionless one. Data unit: segment for TCP, datagram for UDP. Protocols: TCP, UDP.
- Layer 5, Session - establishes, manages, synchronises and terminates sessions between applications. Provides dialogue control and checkpointing so that a long transfer can resume after a failure. Protocols: NetBIOS, RPC.
- Layer 6, Presentation - translates between the application's data format and the network's. Handles character-set conversion, data representation, encryption and decryption, and compression. Formats: JPEG, ASCII, TLS operates around here.
- Layer 7, Application - the layer the user's software interacts with, providing network services to applications. Protocols: HTTP, FTP, SMTP, DNS, SSH, Telnet, SNMP.

Points worth adding

- Mnemonic from the bottom up: Please Do Not Throw Sausage Pizza Away.
- Encapsulation: as data descends, each layer adds its own header, and the receiving side removes them in reverse order as the data ascends.
- Mapping to TCP/IP: OSI layers 1 and 2 correspond to the TCP/IP network access layer, layer 3 to the internet layer, layer 4 to the transport layer, and layers 5, 6 and 7 collapse into the TCP/IP application layer.

Q8 (5 marks)

Differentiate between TCP and UDP protocols.

▼ **ANSWER BELOW** ▼

Stop here if you are still attempting this question.

Answer to Q8

Both are transport-layer protocols that carry data between processes, identified by port numbers. They differ in what guarantees they provide, and therefore in what they cost.

TCP - Transmission Control Protocol

- Connection oriented. A connection is set up by the three-way handshake, SYN, SYN-ACK, ACK, before any data flows, and closed with FIN packets afterwards.
- Reliable. Every segment is acknowledged; unacknowledged segments are retransmitted after a timeout.
- Ordered. Sequence numbers let the receiver reassemble segments in the order sent, even if the network delivers them out of order.
- Flow control through a sliding window, so a fast sender cannot overwhelm a slow receiver.
- Congestion control through slow start, congestion avoidance, fast retransmit and fast recovery, so TCP backs off when the network is loaded.
- Header size 20 bytes minimum, up to 60 with options. Heavier.
- Byte-stream oriented: message boundaries are not preserved, so the application must delimit its own messages.
- Slower, because of handshaking, acknowledgements and retransmission delay.
- Used by HTTP and HTTPS, FTP, SMTP, IMAP, POP3, SSH and Telnet - anywhere loss is unacceptable.

UDP - User Datagram Protocol

- Connectionless. A datagram is simply sent; there is no handshake and no connection state.
- Unreliable, best effort. No acknowledgement and no retransmission, so datagrams may be lost silently.
- Unordered. Datagrams may arrive in any order, and the application must cope.
- No flow control and no congestion control, so UDP will happily flood a link.
- Header size a fixed 8 bytes - source port, destination port, length, checksum. Very light.
- Message oriented: each datagram is delivered whole, preserving boundaries.
- Fast, with minimal latency and overhead.
- Supports broadcast and multicast, which TCP cannot.
- Used by DNS, DHCP, TFTP, SNMP, voice and video streaming, and online games.

How to choose

Choose TCP when every byte must arrive correctly and in order, and a little delay is acceptable - file transfer, web pages, email. Choose UDP when timeliness matters more than completeness, so that a late packet is worse than a lost one - live audio and video, and fast lookups such as DNS. Modern protocols such as QUIC, which underlies HTTP/3, build reliability on top of UDP in order to gain control over congestion and latency while avoiding TCP's handshake cost.

Q9 (5 marks)

Explain the difference between the GET and POST methods in HTML forms.

▼ **ANSWER BELOW** ▼

Stop here if you are still attempting this question.

Answer to Q9

GET and POST are HTTP request methods, selected in the method attribute of a form element. They differ in where the form data travels and in what the request is understood to mean.

GET

- Form data is appended to the URL as a query string after a question mark, with fields separated by ampersands: `/search?q=database&page=2`.
- The data is therefore visible in the address bar, stored in browser history, and written into server and proxy logs.
- Length is limited, because URLs are limited in practice to roughly 2000 characters.
- Only ASCII data can be sent; binary content and file uploads are not possible.
- The request is bookmarkable and shareable, and the resulting page can be cached by the browser and by intermediaries.
- Defined as safe and idempotent: it should only retrieve data, and repeating it should have no additional effect. A browser will re-issue it on refresh or on the back button without warning.
- Appropriate for search queries, filters, pagination and any read-only retrieval.
- In PHP the values arrive in the `$_GET` superglobal array.

POST

- Form data is sent in the body of the HTTP request, not in the URL.
- The data is not visible in the address bar, not kept in history, and not normally written to logs.
- There is no practical size limit, subject only to server configuration.
- Any data type may be sent, including binary, so file upload requires POST with `enctype` set to `multipart/form-data`.
- The request cannot be bookmarked and is not cached by default.
- Neither safe nor idempotent: it is understood to change state on the server, so repeating it may create duplicate records. Browsers warn before re-submitting.
- Appropriate for login, registration, payment, file upload, and any create, update or delete operation.
- In PHP the values arrive in the `$_POST` superglobal array.

Example

```
<!-- visible in the URL, safe to repeat -->
<form action="/search" method="get">
  <input type="text" name="q">
  <button type="submit">Search</button>
</form>

<!-- sent in the body, changes state -->
<form action="/login" method="post">
  <input type="text" name="user">
  <input type="password" name="pass">
  <button type="submit">Log in</button>
</form>
```

A common misconception is that POST is secure. It only hides data from the address bar and the history; the body still travels in plain text unless HTTPS is used. Encryption comes from TLS, not from the choice of method. What is true is that a password sent by GET would be permanently recorded in history and server logs, which is why POST is mandatory for credentials.

Q10 (5 marks)

What are the fundamental syntax rules and the use of variables in PHP scripting?

▼ **ANSWER BELOW** ▼

Stop here if you are still attempting this question.

Answer to Q10

Fundamental syntax rules

- PHP code is enclosed in the tags `<?php ... ?>`. Anything outside the tags is emitted to the browser unchanged. In a file that contains only PHP, the closing tag is conventionally omitted to avoid emitting stray whitespace.
- Every statement ends with a semicolon. The last statement before a closing tag may omit it.
- PHP is a server-side interpreted language: the server executes the code and sends the resulting HTML to the browser, so the client never sees the source.
- Comments are `//` or `#` to the end of the line, and `/* ... */` for a block.
- Case sensitivity is mixed, and this is a common examination point. Variable names are case sensitive, so `$name` and `$Name` are different. Keywords, function names and class names are case insensitive, so `ECHO`, `Echo` and `echo` all work.
- Output is produced with `echo`, which takes one or more arguments, or `print`, which takes one and returns a value. `print_r` and `var_dump` are used for inspecting arrays and objects.
- Strings in double quotes interpolate variables and interpret escape sequences; strings in single quotes are literal. Concatenation uses the dot operator.
- The file extension must be `.php` for the server to process it.

Variables

- Every variable name begins with a dollar sign, followed by a letter or underscore, then any number of letters, digits or underscores. A name may not begin with a digit.
- No declaration is needed and no type is stated. A variable is created by assignment, and PHP is dynamically and weakly typed, so a variable may hold an integer at one moment and a string at the next.
- The data types are integer, float, string, boolean, array, object, NULL and resource.
- Assignment is by value by default. Prefixing the source with an ampersand assigns by reference, so both names refer to the same value.
- Scope: a variable declared outside a function is global but is not visible inside a function unless imported with the `global` keyword or accessed through `$GLOBALS`. A variable declared inside a function is local. The `static` keyword makes a local variable retain its value between calls.
- Constants are defined with `define()` or the `const` keyword, take no dollar sign, and cannot be changed.
- Superglobals are available in every scope: `$_GET`, `$_POST`, `$_REQUEST`, `$_SESSION`, `$_COOKIE`, `$_FILES`, `$_SERVER`, `$_ENV` and `$GLOBALS`.

Example

```
<?php
$name = "Ezekiel";           // string
$marks = 87;                 // integer
$rate = 4.75;                // float
$pass = true;                // boolean

// double quotes interpolate, single quotes do not
echo "Name: $name\n";        // Name: Ezekiel
```

```
echo 'Name: $name';           // Name: $name

// concatenation
echo "Total: " . ($marks + 3) . "\n";

// constant
define("MAX_MARKS", 100);
echo "Percentage: " . ($marks / MAX_MARKS * 100) . "%";

function counter() {
    static $n = 0;           // retains its value between calls
    $n++;
    return $n;
}
?>
```

Variable variables are a PHP peculiarity worth knowing: if `$a = "marks"` then `$$a` refers to the variable named `$marks`.

Section B - Q11 to Q15 - 10 marks each (answer any 3)

Q11 (10 marks)

Explain the Round Robin CPU scheduling algorithm using an example. What are its advantages and disadvantages?

▼ ANSWER BELOW ▼

Stop here if you are still attempting this question.

Answer to Q11

Principle

Round Robin is a preemptive scheduling algorithm designed for time-sharing systems. It is first-come-first-served with preemption added. Every process is given the CPU for at most one fixed interval called the time quantum or time slice. If the process finishes within its quantum it releases the CPU voluntarily; if not, a timer interrupt fires, the process is preempted, its context is saved, and it is placed at the tail of the ready queue while the next process runs.

- The ready queue is a circular FIFO queue, so every process gets its turn in rotation - hence the name.
- No process waits longer than $(n-1) \times q$ time units for its next turn, where n is the number of processes, which is why the algorithm is starvation free.

Worked example

Four processes, all arriving at time 0, with a time quantum of 4 ms.

Process	Burst time
P1	10
P2	4
P3	6
P4	8

Gantt chart:

P1	P2	P3	P4	P1	P3	P4	P1	
0	4	8	12	16	20	22	26	28

Explanation of the trace: P1 runs 0-4 and is preempted with 6 ms left. P2 runs 4-8 and completes exactly. P3 runs 8-12, preempted with 2 ms left. P4 runs 12-16, preempted with 4 ms left. P1 runs 16-20, preempted with 2 ms left. P3 runs 20-22 and completes. P4 runs 22-26 and completes. P1 runs 26-28 and completes.

Process	Burst	Completion	Turnaround (CT)	Waiting (CT - AT)	(TAT - Burst)
P1	10	28	28	18	
P2	4	8	8	4	
P3	6	22	22	16	
P4	8	26	26	18	

- Average turnaround time = $(28 + 8 + 22 + 26) / 4 = 21$ ms.
- Average waiting time = $(18 + 4 + 16 + 18) / 4 = 14$ ms.

Advantages

- Fair. Every process receives an equal share of CPU time, and no process is favoured.
- Starvation free. Every process is guaranteed the CPU within a bounded time, unlike priority scheduling or shortest job first.
- Good response time for short and interactive processes, which is what makes it suitable for time-sharing and for interactive systems.
- Simple to understand and to implement - a FIFO queue and a timer interrupt.
- Predictable. The maximum waiting time for the next turn can be calculated in advance, which matters for soft real-time work.

- No prior knowledge of burst times is needed, unlike SJF or SRTF, which need an estimate that cannot be known reliably.

Disadvantages

- Higher average turnaround time than shortest job first for the same workload, because long processes are repeatedly interrupted.
- Context-switching overhead. Every preemption costs time to save and restore state and to flush pipelines and cache, and that time is pure overhead doing no useful work.
- Performance depends critically on the choice of quantum, and choosing it well is the central difficulty.
- Treats all processes as equal, so it cannot express priority - an important system process gets no more CPU than a trivial one.
- Poor for processes with very different burst times, and not suitable for hard real-time systems, which need deadline-driven scheduling.

Choosing the time quantum

- If the quantum is too large, preemption rarely happens and Round Robin degenerates into first-come-first-served, losing its responsiveness.
- If the quantum is too small, the CPU spends a large fraction of its time context switching rather than executing, and throughput collapses.
- The usual rule of thumb is that the quantum should be large enough that about eighty per cent of CPU bursts finish within it, which in practice puts it in the range of ten to a hundred milliseconds.

A practical refinement is the multilevel feedback queue, which uses Round Robin within each priority level and moves a process to a lower-priority queue with a longer quantum when it uses its whole slice. That combines Round Robin's fairness with an approximation of priority and of shortest job first.

Q12 (10 marks)

Discuss the different types of network topologies, highlighting their key features, advantages, and disadvantages.

▼ ANSWER BELOW ▼

Stop here if you are still attempting this question.

Answer to Q12

Topology is the arrangement of the nodes and links of a network. Physical topology is the actual layout of the cabling; logical topology is the path the data follows, and the two need not be the same.

1. Bus topology

- Feature: all devices connect to a single shared backbone cable, terminated at both ends to prevent signal reflection. Only one device may transmit at a time, so a media access method such as CSMA/CD is needed.
- Advantages: least cable and lowest cost, simple to install, easy to extend for small networks.
- Disadvantages: the backbone is a single point of failure, so a break brings the whole network down; performance degrades sharply as devices are added, because of collisions; fault isolation is difficult; and the cable length is limited. Largely obsolete.

2. Star topology

- Feature: every device connects by its own cable to a central hub or switch, and all traffic passes through that centre. This is the dominant topology in modern LANs, wired and wireless.
- Advantages: failure of one cable or node affects only that node; easy to add or remove devices; simple fault detection and isolation; with a switch, each link is a separate collision domain, so performance is high.
- Disadvantages: the central device is a single point of failure for the whole network; more cable is needed than for a bus; and performance depends on the capacity of the central switch.

3. Ring topology

- Feature: each device connects to exactly two neighbours, forming a closed loop. Data travels in one direction, or in both in a dual ring, and each node regenerates the signal as it passes it on. Access is often controlled by token passing.
- Advantages: no collisions, because access is orderly; each node acts as a repeater, so the signal can travel further; performance is predictable under heavy load.
- Disadvantages: a single break in the ring can stop all traffic, though a dual ring such as FDDI survives one break; adding or removing a node disrupts the ring; and a packet may traverse many intermediate nodes, increasing latency.

4. Mesh topology

- Feature: devices are interconnected directly. In a full mesh every node connects to every other, requiring $n(n-1)/2$ links for n nodes; a partial mesh connects only the important nodes.
- Advantages: highly fault tolerant, since multiple paths exist between any pair; no single point of failure; traffic is distributed, so there is no central bottleneck; privacy, because a link carries only its own pair's traffic.
- Disadvantages: cost and cabling grow quadratically, installation and reconfiguration are complex, and each node needs many interfaces. Used for network backbones, for wireless mesh, and wherever reliability outweighs cost.

5. Tree, or hierarchical, topology

- Feature: groups of star networks connected to a hierarchical backbone, giving a root, intermediate and leaf structure. This is how most campus and enterprise networks are actually built, with core, distribution and access layers.
- Advantages: scales well, allows management by segment, and fault isolation is straightforward within a branch.
- Disadvantages: failure of a higher-level node disconnects its whole subtree, cabling is extensive, and configuration is more complex.

6. Hybrid topology

- Feature: any deliberate combination of the above, for example a star-bus or star-ring. Nearly every large real network is a hybrid.
- Advantages: each segment can use whichever topology suits it, so the design is flexible and scalable.
- Disadvantages: complex design and management, and higher cost.

Comparison summary

Topology	Cost	Reliability	Ease of fault isolation	Scalability
Bus	lowest	poor	poor	poor
Star	moderate	good*	very good	good
Ring	moderate	moderate	moderate	moderate
Mesh	highest	excellent	good	poor (cost)
Tree	high	good	good	excellent

* except for the central device

How a topology is chosen

The decision balances cost, expected reliability, the number of nodes and the anticipated growth, the traffic pattern, and the ease of maintenance. In practice the switched star, aggregated into a tree, has won for access networks because it combines low cost per node with good fault isolation, while mesh is reserved for backbones and for wireless deployments where cabling is not the constraint.

Q13 (10 marks)

Describe the phases of website development, from planning to maintenance.

▼ **ANSWER BELOW** ▼

Stop here if you are still attempting this question.

Answer to Q13

Website development follows a life cycle of its own, similar in shape to the SDLC but with distinct emphases on content, visual design and search visibility.

1. Planning and requirement gathering

- Establish the purpose of the site - to inform, to sell, to serve an application - and define the measurable goals against which success will be judged.
- Identify the target audience and build user personas, and study competitor sites.
- Decide the scope: the feature list, the number of pages, whether a content management system is needed, and which languages must be supported.
- Fix the technical approach - static site, CMS, or custom application - the hosting model, and the budget and timeline.
- Output: a requirements document and a project plan.

2. Information architecture and content planning

- Organise the content into a hierarchy, produce a sitemap, and define the navigation and URL structure.
- Plan the content itself - text, images, video - and decide who will supply it. Content is very often the item that delays a website project.
- Output: sitemap and content inventory.

3. Design

- Wireframes first, establishing layout and hierarchy in grey boxes without visual styling, so that structure can be judged independently of aesthetics.
- Then mockups and a visual design system: colour palette, typography, spacing scale, iconography and imagery, consistent with the brand.
- Design responsively for mobile, tablet and desktop widths, and design the interactive states - hover, focus, active, disabled, loading, empty and error.
- Build a clickable prototype for review, and get sign-off before coding begins.
- Output: approved mockups, prototype and style guide.

4. Development

- Front end: mark up the pages semantically in HTML, style them with CSS, and add behaviour with JavaScript, working from the approved designs. Ensure responsiveness and accessibility.
- Back end: implement server-side logic, the database schema, authentication, business rules and APIs, using a stack such as PHP with MySQL, Node.js, Django or Laravel.
- Integrate a CMS if required, so that non-technical staff can edit content.
- Add the content, and configure third-party integrations such as payment gateways, analytics and email.
- Work under version control, with code review, in a development environment separate from production.
- Output: a working site in a staging environment.

5. Testing

- Functional testing of every form, link, workflow and calculation.

- Cross-browser and cross-device testing, including older browsers if the audience requires it.
- Responsive testing at every breakpoint, and on real devices as well as emulators.
- Performance testing - page weight, image optimisation, caching, Core Web Vitals, load testing.
- Accessibility testing against WCAG, with a screen reader and with keyboard-only navigation.
- Security testing for injection, cross-site scripting, cross-site request forgery, insecure direct object references and weak authentication.
- SEO checks: titles, meta descriptions, heading structure, alt text, structured data, sitemap.xml, robots.txt, canonical URLs.
- User acceptance testing by the client.

6. Deployment, or launch

- Provision the production hosting, configure the domain and DNS, and install a TLS certificate so the site is served over HTTPS.
- Deploy the code and migrate the database and media, ideally through an automated pipeline.
- Set up backups, monitoring and error reporting before announcing the launch, not after.
- Submit the sitemap to search engines, and set up redirects from the old URLs if the site is a replacement - omitting these is the classic way to lose accumulated search ranking.
- Verify analytics is recording, and re-test the critical paths in production.

7. Maintenance and continuous improvement

- Apply security patches to the CMS, plugins, libraries and server. An unpatched CMS is the most common cause of a compromised website.
- Verify backups by restoring them, monitor uptime and performance, and renew certificates and the domain.
- Update content, fix defects reported by users, and improve based on analytics - what pages people actually visit, where they abandon, what they search for.
- Run A/B tests on important pages, and periodically revisit SEO and accessibility.
- Plan a redesign when the design or the technology stack has aged, typically after three to five years.

The essential difference from general software: a website is never finished. Maintenance and iteration are the normal state, not a closing phase, and the budget should reflect that from the outset.

Q14 (10 marks)

Compare and contrast knowledge-level learning and symbol-level learning in Artificial Intelligence.

▼ **ANSWER BELOW** ▼

Stop here if you are still attempting this question.

Answer to Q14

This distinction, due to Allen Newell's analysis of levels of description in intelligent systems, separates learning that changes what a system knows from learning that changes how efficiently it uses what it already knows. It is the difference between acquiring competence and acquiring speed.

Knowledge-level learning

- Definition: learning that increases the deductive closure of the system's knowledge - that is, the set of conclusions it is capable of reaching. After learning, the system can answer questions it genuinely could not answer before.
- What changes: the content of the knowledge base. New facts, new rules, new concepts, new relationships are added.
- Nature: the system becomes more competent. It gains knowledge that was not derivable from what it held previously.
- Typical methods: inductive learning, that is generalising from examples; learning by being told; learning by discovery; concept formation; rule induction from data.
- Example: a medical diagnosis system that is given, or induces from case records, the new rule that a particular combination of symptoms indicates a particular disease. It can now diagnose a condition it previously could not.
- Evaluation: measured by accuracy, coverage and generalisation to unseen cases.

Symbol-level learning

- Definition: learning that improves the efficiency with which the system uses knowledge it already possesses. The deductive closure is unchanged - nothing new is knowable - but conclusions are reached faster or with less search.
- What changes: the representation and organisation of the knowledge, or the control strategy that searches it.
- Nature: the system becomes more competent in speed rather than in scope. It is a performance optimisation.
- Typical methods: Explanation-Based Learning, which generalises the explanation of a single solved example into a reusable macro-rule; chunking, as in the SOAR architecture; caching or memoising of results; rule reordering; macro-operator formation; index construction.
- Example: a planner solves a goal through a long chain of inference, then stores the derived result as a single rule. The next occurrence is answered immediately instead of being re-derived. Nothing new was learned in the logical sense; the derivation was merely made cheap.
- Evaluation: measured by search time, number of inference steps, and memory used.

Points of comparison

- Knowledge content: extended at the knowledge level, unchanged at the symbol level.
- What is gained: new capability versus greater speed.
- Data requirement: knowledge-level learning usually needs many examples in order to generalise safely; symbol-level learning can profit from a single worked example, provided a domain theory is available to explain it.

- Prior knowledge: knowledge-level learning may start from very little; Explanation-Based Learning requires a reasonably complete domain theory, because it must be able to explain why the example worked.
- Risk: knowledge-level learning risks acquiring an incorrect generalisation, that is over-fitting or a wrong rule; symbol-level learning risks the utility problem, where so many macro-rules accumulate that the cost of matching them outweighs the saving they were meant to produce.
- Correctness: symbol-level learning is deductively sound, since it only re-expresses what was already entailed; inductive knowledge-level learning is not sound, because a generalisation from examples may be wrong.

How they relate

The two are complementary rather than competing, and a capable system needs both. Knowledge-level learning determines what the system can do at all; symbol-level learning determines whether it can do it quickly enough to be useful. A system that only learns at the symbol level becomes a fast expert on a fixed domain and can never extend its reach. A system that only learns at the knowledge level accumulates knowledge it becomes progressively slower at using, because the search space grows with the knowledge base. This is why architectures such as SOAR pair a learning mechanism for new knowledge with chunking for efficiency.

Q15 (10 marks)

Explain the k-Nearest Neighbors (k-NN) algorithm for classification. What factors influence its performance?

▼ **ANSWER BELOW** ▼

Stop here if you are still attempting this question.

Answer to Q15

The algorithm

k-NN is a supervised, instance-based, non-parametric classification method. It is called a lazy learner because it has no training phase in the usual sense: it simply stores the training set, and all the work happens at prediction time. The underlying assumption is that similar inputs have similar labels.

1. Store all training examples with their feature vectors and class labels. This is the whole of training.
2. To classify a new instance, compute the distance from it to every stored training example.
3. Select the k training examples with the smallest distances - the k nearest neighbours.
4. Assign the class that is most common among those k neighbours, that is take a majority vote. For regression, take the mean of their values instead.

Distance measures

- Euclidean distance, the usual choice for continuous features: the square root of the sum of squared coordinate differences.
- Manhattan distance, the sum of absolute differences, which is more robust to outliers and often better in high dimensions.
- Minkowski distance, the general form with parameter p; $p = 1$ gives Manhattan and $p = 2$ gives Euclidean.
- Hamming distance for categorical features, and cosine similarity for text or other sparse high-dimensional vectors where orientation matters more than magnitude.

Worked illustration

With $k = 5$ and a new point whose five nearest neighbours have labels A, A, B, A, B, the majority is A with three votes, so the prediction is A. Note that with $k = 3$ and neighbours A, B, B the prediction would instead be B, which shows how sensitive the result can be to k.

Factors that influence performance

- The value of k. A small k, especially $k = 1$, gives a highly flexible boundary that fits noise, that is low bias and high variance. A large k smooths the boundary and may ignore genuine local structure, that is high bias and low variance. In the limit $k = n$ every prediction is the majority class. k is normally chosen by cross-validation, and an odd k is preferred for two-class problems so that votes cannot tie.
- Feature scaling. This is the most common practical mistake. Distance is dominated by whichever feature has the largest numeric range, so a salary in the tens of thousands will overwhelm an age in tens. Features must be normalised to a common range or standardised to zero mean and unit variance before distances are computed.
- The curse of dimensionality. As the number of features grows, points become nearly equidistant from one another and the notion of a nearest neighbour loses meaning. The data also becomes sparse, so the required sample size grows exponentially. Dimensionality reduction with PCA, or deliberate feature selection, is usually necessary.
- Irrelevant and redundant features. Unlike a decision tree, k-NN has no built-in mechanism for ignoring a useless feature - every feature contributes to the distance. Feature selection, or feature weighting, materially improves accuracy.

- The choice of distance metric, which must match the nature of the data - Euclidean for continuous, Hamming for categorical, cosine for text.
- Class imbalance. If one class dominates the training set it will dominate the neighbourhood vote regardless of true similarity. Remedies are distance-weighted voting, where a nearer neighbour counts for more, and resampling.
- Noise and outliers, to which small values of k are especially vulnerable. A larger k averages the noise away.
- The size and representativeness of the training set. More data generally improves k -NN, since it can only interpolate between examples it holds, but more data also makes prediction slower.
- Computational cost at prediction time. A naive search is $O(n \times d)$ per query, which is prohibitive for large n . Spatial index structures such as a k -d tree or a ball tree reduce this for low dimensions, and approximate nearest-neighbour methods such as locality-sensitive hashing are used for high dimensions.

Advantages and disadvantages

- Advantages: simple to understand and to implement, no training cost, naturally handles multi-class problems, makes no assumption about the shape of the data distribution, and adapts immediately when new examples are added.
- Disadvantages: slow and memory-hungry at prediction time, sensitive to scaling and to irrelevant features, degrades badly in high dimensions, and offers no interpretable model - there is nothing to inspect except the data itself.

Section C - Q16 to Q19 - 15 marks each (answer any 2)

Q16 (15 marks)

Discuss the different memory management techniques used in operating systems, including paging and segmentation.

▼ ANSWER BELOW ▼

Stop here if you are still attempting this question.

Answer to Q16

Memory management is the operating-system function that decides which process occupies which part of main memory, translates the addresses a program uses into real ones, and creates the illusion of more memory than physically exists. Its goals are efficient utilisation, protection between processes, controlled sharing, and relocation.

1. Contiguous allocation

- Single contiguous allocation - the simplest scheme. Memory is divided into one region for the operating system and one for the single user program. No multiprogramming, and memory is badly wasted. Used in early systems such as MS-DOS.
- Fixed, or static, partitioning - memory is divided in advance into a fixed number of partitions, each holding one process. The degree of multiprogramming is limited to the number of partitions. Its defect is internal fragmentation: a process smaller than its partition wastes the remainder, and that space cannot be given to anyone else.
- Variable, or dynamic, partitioning - a partition is created to the exact size of each process as it arrives. This removes internal fragmentation but produces external fragmentation: as processes come and go, the free memory is left as many small holes, so a large request may fail even though the total free space is sufficient.
- Allocation strategies for variable partitioning: first fit takes the first hole large enough and is fastest; best fit takes the smallest adequate hole and tends to leave tiny useless slivers; worst fit takes the largest hole; next fit resumes searching from the last position.
- Compaction relocates processes to gather the free space into one block. It cures external fragmentation but is expensive and requires execution-time address binding.

2. Paging

- Concept: physical memory is divided into fixed-size blocks called frames, and the logical address space into blocks of the same size called pages. Any page may be placed in any free frame, so the process need not be contiguous in physical memory. This eliminates external fragmentation entirely.
- Address translation: the logical address is split into a page number p and an offset d . The page number indexes the page table to obtain the frame number f , and the physical address is formed by concatenating f with d . The page size is a power of two so that the split is a matter of bit positions rather than arithmetic.

Logical address: | page number p | offset d |

page table[p] -> frame number f

Physical address: | frame number f | offset d |

- The page table is per process and is held in memory; the page-table base register points to it. Each entry also holds a valid/invalid bit, protection bits, a modified or dirty bit and a referenced bit.
- Because the page table is in memory, every logical reference would need two physical accesses. The Translation Lookaside Buffer, a small fully-associative hardware cache of recent translations, removes this cost for the great majority of references. Effective access time = hit ratio $\times$ (TLB time + memory time) + miss ratio $\times$ (TLB time + 2 $\times$ memory time).

- Internal fragmentation remains, but only in the last page of a process, so on average half a page per process - negligible.
- Large address spaces make a single flat page table impractical, so real systems use hierarchical, or multilevel, page tables, hashed page tables, or inverted page tables.
- Paging is invisible to the programmer, which is both its convenience and its limitation - it imposes no logical structure on the address space.

3. Segmentation

- Concept: the logical address space is divided into segments that correspond to the programmer's own logical units - code, global data, heap, stack, a particular function, an array. Segments are of different sizes, which is the essential difference from pages.
- Address translation: a logical address is a pair $\langle \text{segment number } s, \text{offset } d \rangle$. The segment table gives a base and a limit for each segment. The offset is checked against the limit, and if it is within bounds the physical address is $\text{base} + d$; otherwise a trap is raised.

Logical address: $\langle \text{segment } s, \text{offset } d \rangle$

```
segment table[s] -> (base, limit)
```

```
if d < limit -> physical = base + d
else        -> addressing error trap
```

- Advantages: it matches the programmer's view, so protection and sharing can be applied per logical unit. A code segment can be marked read-only and execute-only, and a shared library segment can be mapped into several processes. A segment can also grow, which suits a stack or a heap.
- Disadvantage: because segments vary in size, segmentation suffers from external fragmentation exactly as variable partitioning does, and requires compaction.

4. Segmented paging - the combination

- Each segment is itself paged, so the address becomes $\langle \text{segment number}, \text{page number}, \text{offset} \rangle$. The segment table entry points to a page table for that segment.
- This takes the logical structure, protection and sharing of segmentation and the freedom from external fragmentation of paging. It is the scheme used by the Intel x86 architecture and, in effect, by modern operating systems, which keep segmentation minimal and rely on paging.

5. Virtual memory and demand paging

- Concept: not all of a process needs to be in memory at once. Pages are loaded only when referenced, so a process may have a logical address space larger than physical memory, and more processes can be resident, raising the degree of multiprogramming and CPU utilisation.
- A reference to a page whose valid bit is clear causes a page fault. The operating system finds the page on the backing store, selects a victim frame, writes the victim back if it is dirty, reads the required page in, updates the page table and restarts the faulting instruction.
- Page-replacement algorithms: FIFO, simple but subject to Belady's anomaly, where more frames can produce more faults; Optimal, which replaces the page not needed for the longest time and is unimplementable but serves as a benchmark; LRU, which approximates Optimal well and is

implemented with counters, a stack, or the second-chance and clock algorithms using the reference bit; LFU and MFU, which are used less often.

- Frame allocation may be equal, proportional to process size, or priority based, and replacement may be local, within the process's own frames, or global, across all processes.
- Thrashing occurs when a process does not have enough frames to hold its working set, so it spends more time servicing page faults than executing. CPU utilisation collapses, and the scheduler's instinct to raise the degree of multiprogramming makes it worse. The cures are the working-set model, which allocates frames according to the pages a process has actually referenced in a recent window, and page-fault frequency control.
- Locality of reference is the property that makes all of this work: a process references a small subset of its pages during any phase of execution.

6. Swapping

A whole process may be moved out of main memory to the backing store and later brought back. Swapping frees memory in bulk and is used to relieve pressure or to suspend a process, but the transfer cost is high, and modern systems prefer page-level demand paging to whole-process swapping.

Comparison of paging and segmentation

	Paging	Segmentation
Block size	fixed	variable
Divided by	hardware	the programmer/compiler
Logical meaning	none	matches program units
Fragmentation	internal (small)	external
Table per proc.	page table	segment table
Table entry	frame number	base + limit
Protection unit	page	logical segment
Sharing	per page, awkward	per segment, natural
Compaction	not needed	needed

Q17 (15 marks)

Explain the various cost estimation methods used in software project planning. Which method do you think is most reliable and why?

▼ ANSWER BELOW ▼

Stop here if you are still attempting this question.

Answer to Q17

Software cost estimation predicts the effort, usually in person-months, and from it the schedule and the cost, needed to deliver a software product. It matters because it drives bidding, staffing and commitment, and because software projects have a long history of overrunning - the classic finding is that estimates made early are routinely optimistic by a factor of two or more.

1. Expert judgement

- Method: one or more experienced practitioners estimate from their own knowledge of similar work.
- Delphi technique: several experts estimate independently and anonymously, the estimates are summarised and circulated, and the process is repeated until convergence. Anonymity prevents the loudest or most senior voice from dominating. Wideband Delphi adds a group discussion between rounds.
- Advantages: fast, cheap, needs no historical database, and can take account of factors no formula captures - a difficult client, an unfamiliar technology, a team's actual morale.
- Disadvantages: subjective and unrepeatable; quality depends entirely on the expert; systematically optimistic, both from the planning fallacy and from pressure to win the work; and hard to justify to a customer.

2. Estimation by analogy

- Method: find one or more completed projects similar in size, domain and technology, take their actual costs, and adjust for the known differences.
- Advantages: grounded in what actually happened rather than in what someone believes; relatively easy to explain and defend; accurate when a genuinely comparable project exists.
- Disadvantages: requires a well-kept historical database, which many organisations do not have; depends on the analogy really being close; and offers no help for a genuinely novel project.

3. Bottom-up, or decomposition, estimation

- Method: decompose the work with a WBS until each work package is small, estimate each package, and sum the results, adding integration, management and testing effort.
- Advantages: usually the most accurate technique available, because small pieces are far easier to judge than a whole; errors in individual packages tend to cancel; produces an estimate the team owns, which improves commitment; and gives visibility into where the effort actually goes.
- Disadvantages: time consuming; requires a fairly complete design or at least a decomposition, so it cannot be used at the very start; and tends to omit work that appears in no package - integration, rework, meetings, environment setup.

4. Top-down estimation

- Method: estimate the project as a whole from its overall properties, then apportion the total among phases and components using historical percentages.
- Advantages: quick, usable very early, and it naturally accounts for system-level activities such as integration and project management.
- Disadvantages: much less accurate; hides low-level technical difficulties; and gives the team little to work with.

5. Algorithmic, or parametric, models

- Method: apply a formula calibrated on historical data, with size as the primary driver and a set of adjustment factors.
- Lines of code, or KLOC, as the size measure - simple and directly countable, but language dependent, impossible to know accurately in advance, and it perversely rewards verbose code.
- COCOMO, the Constructive Cost Model of Boehm, 1981. Basic COCOMO: $\text{Effort} = a \times (\text{KLOC})^b$ person-months, with coefficients depending on project class - organic for small familiar projects, semi-detached for medium, embedded for tightly constrained. Intermediate COCOMO multiplies the result by fifteen cost drivers covering product, hardware, personnel and project attributes. Detailed COCOMO applies the drivers per phase. COCOMO II updates the model for modern practice, adds reuse and scale factors, and allows function points as the size input.
- Note that the exponent b exceeds 1, which encodes diseconomy of scale: doubling the size more than doubles the effort, because communication paths grow faster than the team.
- Function Point Analysis, due to Albrecht. Count external inputs, external outputs, external inquiries, internal logical files and external interface files; weight each by complexity to get unadjusted function points; then apply a value adjustment factor derived from fourteen general system characteristics. Function points are technology and language independent, and can be counted from the requirements before any code exists - their advantage over KLOC. Their weakness is that counting requires training and involves judgement, and they suit data-intensive business systems better than algorithm-intensive or real-time ones.
- Putnam's SLIM model, based on the Rayleigh manpower distribution curve, relates size, effort and schedule and shows that compressing a schedule raises effort sharply.
- Advantages of algorithmic models: objective, repeatable, defensible, and they permit sensitivity analysis - what happens if the team is less experienced, or the reliability requirement rises.
- Disadvantages: they must be calibrated against local historical data or they are badly wrong; they depend on a size estimate that is itself uncertain, so an error in size propagates and is amplified; and they cannot represent factors outside the model.

6. Other techniques

- Three-point estimation, from PERT: $\text{Expected} = (\text{Optimistic} + 4 \times \text{Most likely} + \text{Pessimistic}) / 6$, which acknowledges uncertainty rather than pretending to a single number, and yields a standard deviation as well.
- Planning poker and story points, used in Agile. The team estimates relative size rather than absolute duration, then converts to time using measured velocity, that is story points completed per sprint. This works well because relative judgement is more reliable than absolute, and because velocity is empirical rather than assumed.
- Price-to-win, in which the estimate is set to whatever the customer will accept. It is common, and it is the leading cause of failed projects; it is an estimate only in name.

Which is most reliable, and why

- No single method is reliable in isolation. The most reliable practice is bottom-up decomposition, cross-checked against a parametric model calibrated on the organisation's own historical data, with the difference between the two investigated rather than averaged away.

- Bottom-up is the strongest single technique, for three reasons. First, decomposition converts one impossible judgement into many tractable ones, and independent errors partially cancel rather than accumulate. Second, it forces the estimator to confront the actual work, which exposes tasks that a top-down figure silently omits. Third, the estimate is produced by the people who will do the work, and estimates people own are estimates they meet.
- It needs two corrections to be trustworthy. It must include an explicit allowance for the work that belongs to no work package - integration, rework, defect fixing, meetings, environment and tooling - which is commonly thirty per cent or more of the total. And it must be checked against a top-down or parametric figure, because bottom-up estimates are prone to omission rather than to over-estimation.
- Why the parametric cross-check matters: COCOMO II or function points, calibrated locally, bring historical reality to bear and will flag an estimate that is optimistic overall even when every individual package looks reasonable.
- Two further practices matter more than the choice of method. Express the estimate as a range or a distribution rather than a single number - the cone of uncertainty means that an estimate made at the requirements stage cannot be better than roughly plus or minus fifty per cent, and claiming otherwise is dishonest. And re-estimate at each milestone as uncertainty falls, rather than defending the original figure.

In an Agile context the most reliable predictor is not any of these models but measured velocity: after three or four sprints, the team's own observed throughput forecasts the remainder better than any formula, because it incorporates every local factor automatically.

Q18 (15 marks)

Describe the query processing steps in a relational database management system. Why is query optimization important?

▼ ANSWER BELOW ▼

Stop here if you are still attempting this question.

Answer to Q18

Query processing is the sequence of activities by which a declarative SQL statement is turned into an efficient sequence of operations on stored data, and executed. The user states what is wanted; the system decides how to get it. The steps are the following.

Step 1 - Parsing and translation

- Syntactic analysis: the scanner and parser check that the statement conforms to SQL grammar, and build a parse tree. A syntax error is reported here.
- Semantic analysis, or validation: the system consults the data dictionary, or system catalogue, to confirm that every table and column named actually exists, that the user has the necessary privileges, and that types are compatible. Views are expanded into their defining queries at this point.
- Translation: the validated parse tree is converted into an internal representation, normally a relational algebra expression or a query tree, whose leaves are the base relations and whose internal nodes are operations - selection, projection, join, aggregation. This is what the rest of the pipeline works on, because relational algebra has known equivalence laws.

SQL:

```
SELECT s.name
FROM   Student s, Enrol e
WHERE  s.sid = e.sid AND e.grade = 'A';
```

Relational algebra:

```
PROJECT[s.name] ( SELECT[e.grade='A'] ( Student JOIN[s.sid=e.sid] Enrol ) )
```

Step 2 - Query optimization

- The optimiser generates alternative equivalent execution plans and chooses the one it estimates to be cheapest. Two kinds of decision are involved.
- Logical, or algebraic, transformation - rewriting the relational algebra expression using equivalence rules, without regard to how each operation will be implemented. The key heuristics are: push selections down as close to the leaves as possible, so that rows are discarded before they are joined; push projections down, so that unneeded columns are dropped early; convert a Cartesian product followed by a selection into a join; reorder joins so that the most selective join is performed first, keeping intermediate results small; and eliminate redundant operations and unsatisfiable conditions.
- Physical planning - choosing an access path and an algorithm for each operation. Access path: full table scan, or an index scan using a B+-tree or hash index, or an index-only scan when the index covers all required columns. Join algorithm: nested-loop join, block nested-loop, index nested-loop, sort-merge join, or hash join, each with different cost behaviour. Also whether to materialise an intermediate result or to pipeline it into the next operator, and whether to exploit parallelism.
- Cost estimation - the optimiser assigns each candidate plan an estimated cost, dominated in practice by disk I/O, that is the number of block transfers and seeks, with CPU and, in a distributed system, network cost also counted. The estimates come from statistics kept in the catalogue: the number of rows in each relation, the number of blocks, the number of distinct values in each column, histograms of value distribution, index height and clustering. Selectivity - the fraction of rows a predicate will pass - is estimated from these, and the size of each intermediate result is propagated up the tree.

- Search strategy - the number of possible plans grows factorially with the number of joined relations, so exhaustive enumeration is impossible beyond a handful of tables. Real optimisers use dynamic programming over join orders in the manner of System R, or heuristic and randomised search, or genetic algorithms, and they prune plans that cannot win.
- Two families of optimiser: rule-based, which applies a fixed priority of heuristics and needs no statistics; and cost-based, which estimates and compares. All significant modern systems are cost-based.

Step 3 - Code generation and evaluation

- The chosen logical plan becomes an executable plan - either compiled code or, more commonly, a tree of iterator objects, each implementing open, next and close, the so-called Volcano or iterator model.
- The execution engine runs the plan. Pipelining passes each output row of one operator directly as input to the next, avoiding writing intermediate results to disk. Materialisation writes an intermediate relation to temporary storage, which is necessary for blocking operators such as sorting, or when a result is needed more than once.
- The result is assembled and returned to the application through a cursor, and statistics from the run may be fed back for future planning.

Why query optimization is important

- The performance difference between plans is not marginal, it is categorical. Two logically equivalent plans for the same query can differ by several orders of magnitude - a join of two large tables done as a Cartesian product followed by a filter may examine a billion row pairs, where an index nested-loop join examines a few thousand. On real data this is the difference between milliseconds and hours.
- SQL is declarative, so the user does not, and should not, specify an execution strategy. Someone must choose one, and the optimiser is that someone. Without it, the burden of hand-tuning every query would fall on the application programmer, and data independence - one of the central promises of the relational model - would be lost.
- The best plan depends on the current state of the data, not only on the query text. A predicate that is highly selective when a table is small may match most rows after the table grows, which changes whether an index is worth using. Only a cost-based optimiser working from live statistics can adapt to that, and it re-decides as the data changes without the query being rewritten.
- Disk I/O dominates cost, and the optimiser's principal lever is reducing the volume of data touched - choosing an index instead of a scan, filtering before joining, and keeping intermediate results small. Since a disk access is several orders of magnitude slower than a memory access, saving block transfers is where the gains are.
- Resource sharing and scalability. A database serves many concurrent users, so a wasteful plan consumes memory, CPU, buffer pool and I/O bandwidth that other transactions need. One bad plan can degrade the whole system, not just its own query. Efficient plans therefore raise total throughput, not merely single-query latency.
- It enables the abstraction that makes databases usable. Views, subqueries, and complex joins written for clarity are transformed into efficient forms by the optimiser, so developers can write readable, maintainable SQL instead of contorting queries for performance.

Practical implications worth stating

- Because the optimiser depends on statistics, statistics must be kept current - a stale histogram is a common cause of a sudden plan regression. Systems provide ANALYZE or UPDATE STATISTICS for this.
- Poor plans usually trace to a bad cardinality estimate, most often from correlated predicates, which the optimiser typically assumes to be independent.
- EXPLAIN, or EXPLAIN PLAN, shows the chosen plan and is the primary diagnostic tool. Optimiser hints exist to force a plan, but they should be a last resort, since they freeze a decision that ought to adapt as the data changes.

Q19 (15 marks)

Discuss the process of building a knowledge base in Artificial Intelligence. What are the key challenges involved?

▼ ANSWER BELOW ▼

Stop here if you are still attempting this question.

Answer to Q19

A knowledge base is the component of a knowledge-based or expert system that holds domain knowledge in a form an inference engine can reason over. It is distinct from a database in that it stores not only facts but also rules, relationships and constraints, and supports the derivation of new facts rather than merely their retrieval. Building one is called knowledge engineering.

The process

Step 1 - Identification and scoping

- Define the task the system is to perform and the boundaries of the domain. A narrow, well-bounded domain is a precondition for success; open-ended domains defeat the approach.
- Establish who the users are, what questions the system must answer, and what level of performance would count as adequate.
- Assess feasibility: is there genuine expertise to capture, are experts available, and is the knowledge stable enough to be worth encoding.

Step 2 - Knowledge acquisition

- This is the extraction of knowledge from its sources, and it is the hardest and most time-consuming step - historically the bottleneck of the whole enterprise.
- Sources: human domain experts, textbooks, manuals, standards and regulations, case records and databases, and increasingly machine learning over historical data.
- Techniques with experts: structured and unstructured interviews; protocol analysis, in which the expert thinks aloud while solving a real case; observation of practice; card sorting and laddering to elicit how concepts are organised; and repertory grids.
- Automated and semi-automated acquisition: rule induction from data, text mining, and information extraction from documents.

Step 3 - Conceptualisation and knowledge representation

- Identify the key concepts, the relationships between them, and the granularity at which to model. Build a conceptual model or ontology before committing to a formalism.
- Choose a representation scheme, which is the central design decision because it fixes what can be expressed and what inference is possible.
- Production rules, of the form IF condition THEN action or conclusion. Modular, readable by the expert, and directly usable for explanation. The classic choice for diagnostic systems such as MYCIN. Weak at representing structural or taxonomic knowledge.
- Semantic networks - a labelled graph of nodes for concepts and arcs for relations, with is-a and part-of inheritance. Intuitive, good for taxonomies, but with no standard semantics for quantification or negation.
- Frames - structured records with slots and fillers, defaults and inheritance, well suited to describing stereotypical objects and situations, and the ancestor of object-oriented modelling.
- Logic - propositional and first-order predicate logic, with well-defined semantics and sound inference. Expressive and rigorous, but inference is computationally expensive and classical logic handles uncertainty and exceptions badly.

- Ontologies and description logics, such as OWL and RDF, which give a formal, explicit specification of a shared conceptualisation, support reasoning and are the basis of the Semantic Web and of modern knowledge graphs.
- Scripts, for stereotyped sequences of events, and case bases, for reasoning by analogy from stored cases rather than from generalised rules.

Step 4 - Formalisation and encoding

- Translate the conceptual model into the chosen formalism using a shell or a language - CLIPS, Jess, Drools, Prolog, or an ontology editor such as Protege.
- Keep the knowledge base separate from the inference engine. That separation is the architectural point of an expert system: knowledge can be revised without changing the reasoning mechanism.
- Decide the inference strategy: forward chaining, data-driven, from facts towards conclusions, which suits monitoring and interpretation; or backward chaining, goal-driven, from a hypothesis towards the evidence for it, which suits diagnosis; or a mixture.
- Add uncertainty handling where the domain requires it - certainty factors, Bayesian networks, Dempster-Shafer theory, or fuzzy logic.

Step 5 - Verification, validation and testing

- Verification asks whether the knowledge base is internally sound: are there contradictory rules, redundant or subsumed rules, circular chains, unreachable rules, or gaps in coverage.
- Validation asks whether it is right: run archived cases with known outcomes and compare the system's conclusions with the experts'. Blind comparison against a panel of experts is the strongest form, and is how MYCIN was evaluated.
- Test the explanation facility as well as the conclusions. A right answer that cannot be justified will not be trusted or adopted.

Step 6 - Deployment, maintenance and refinement

- Deploy with a usable interface and integrate with existing systems and data sources.
- Refine iteratively: knowledge engineering is inherently cyclic, and every failing case exposes a missing or wrong rule.
- Maintain the knowledge base as the domain changes - new regulations, new drugs, new products. A knowledge base that is not maintained silently becomes wrong, which is more dangerous than being obviously broken.

Key challenges

- The knowledge acquisition bottleneck - the single greatest obstacle. Extracting knowledge from experts is slow and expensive, and the resulting rate of usable rules per week of effort is low.
- Tacit knowledge. Much genuine expertise is compiled into intuition and is not available to introspection. An expert can recognise a case immediately but cannot state the rule being applied - the phenomenon that expertise is knowing more than you can say. Protocol analysis mitigates this but does not remove it.
- Expert availability and willingness. The best experts are the busiest, and some perceive the project as a threat to their own indispensability.

- Disagreement between experts. Two competent experts may reach different conclusions on the same case, so the knowledge engineer must decide whose judgement to encode, or represent the disagreement explicitly.
- Communication gap. The domain expert does not know the formalism and the knowledge engineer does not know the domain, so misunderstanding is systematic and must be actively checked by feeding the encoded knowledge back to the expert.
- Choosing the representation. Different knowledge needs different formalisms, and a scheme chosen for one kind of knowledge will distort another. Committing early to production rules, for instance, makes taxonomic knowledge awkward to express.
- Uncertainty, vagueness and incompleteness. Real domains contain probabilistic evidence, imprecise terms such as large or elevated, and missing information, none of which classical logic handles gracefully.
- Exceptions and non-monotonic reasoning. Rules in real domains have exceptions, and adding a fact may invalidate a previous conclusion, which classical monotonic logic cannot express. This requires default logic, truth maintenance or belief revision.
- Consistency and scale. As the knowledge base grows, contradictions and unintended interactions between rules become likely, and they are hard to detect. Beyond a few hundred rules the system becomes difficult to understand as a whole.
- The utility problem. Adding knowledge, and adding derived macro-rules, increases matching cost, so at some point more knowledge makes the system slower rather than better.
- Brittleness and the absence of common sense. A knowledge-based system performs well inside its domain and fails without warning just outside it, because it has no general background knowledge to fall back on. Attempts to encode common sense wholesale, notably the Cyc project begun in 1984, illustrate the scale of the difficulty.
- Maintenance cost. Knowledge decays as the world changes, and keeping a large rule base current is a permanent commitment rather than a one-off cost.
- Validation is genuinely hard. There is often no objective ground truth, only expert opinion, which is itself variable.

Modern context, worth adding

Contemporary practice increasingly builds knowledge bases as knowledge graphs, populated semi-automatically by information extraction from text and by embedding-based link prediction, rather than entirely by hand. Large language models now supply the broad common-sense background that hand-built systems always lacked. The result is a hybrid: a curated symbolic knowledge base provides precision, auditability and explanation, while learned components provide coverage and robustness. The two approaches address each other's characteristic weaknesses - symbolic systems are brittle but explainable, learned systems are broad but opaque - and retrieval-augmented generation is the current mainstream way of combining them.

